

DSA FROM SCRATCH TO ADVANCED

Day 8 — Searching in Arrays

Concept → Example → Implementation → Dry Run → Complexity → Pattern → Problem → Solution → Takeaway

C++ | DSA | Interview Preparation

Author: Ritesh

What You'll Learn

This chapter moves from the simplest way to search an array to interview-level binary-search patterns.

ROADMAP

1. Search fundamentals → 2. Linear Search → 3. Binary Search → 4. Complexity → 5. Duplicates & boundaries → 6. Interview problems → 7. Revision

Core checklist

- What is searching?
- Why searching matters
- Linear Search & C++ implementation
- Binary Search & C++ implementation
- Dry runs and complexity analysis
- Duplicate handling and boundary patterns
- Binary Search on Answer
- Interview mistakes and practical tips

What Is Searching?

Searching means finding whether a target element exists in a collection and locating its position when required.

```
Array = [12, 7, 25, 4, 18]  
Target = 25  
Answer = Index 2
```

Think in four parts

Input — the collection **Target** — what we want **Output** — usually an index/value **Not found** — commonly -1

For arrays, the key question is: how much of the search space can I eliminate at each step?

Why Do We Need Searching?

Searching appears everywhere: software systems repeatedly need to locate a record, ID, product, username, or value.

Student record • Product catalog • Username lookup • ID lookup • Database lookup • Array element

Choosing the algorithm matters

Unsorted data → Linear Search

You generally cannot safely discard half of an unsorted array.

Sorted / ordered search space → Binary Search

Ordering gives us information that lets us eliminate large portions of the search space.

KEY INSIGHT: The structure of the data determines the algorithm we can use.

Linear Search — Concept

Linear Search checks elements one by one until the target is found or the array ends.

```
[3, 5, 7, 9, 11]  
Target = 9
```

Visual scan

```
3 → ✗  
5 → ✗  
7 → ✗  
9 → ✓ FOUND
```

It works on both sorted and unsorted arrays. Its main trade-off is that, in the worst case, we may inspect every element.

Linear Search — Algorithm

The algorithm is deliberately simple:

1. Start at index 0
2. Compare the current element with the target
3. If equal, return the index
4. Otherwise move to the next index
5. Continue until the array ends
6. Return -1 if the target is absent

Flow

```
START → i = 0 → arr[i] == target?  
↓ YES → RETURN i  
↓ NO  
i = i + 1  
↓  
i < n ? → YES: repeat  
↓ NO  
RETURN -1
```

Linear Search — C++ Implementation

```
int linearSearch(const vector& arr, int target) {  
    for (int i = 0; i < arr.size(); i++) {  
        if (arr[i] == target) {  
            return i;  
        }  
    }  
    return -1;  
}  
  
vector arr = {3, 5, 7, 9, 11};  
int index = linearSearch(arr, 9);  
// index = 3
```

Line-by-line idea

for scans every valid index. **arr[i] == target** checks the match. Returning immediately avoids unnecessary work after the first match. **-1** represents “not found.”

Use a const reference because the search does not need to modify the array.

Linear Search — Dry Run

```
Array = [10, 25, 30, 45, 60]  
Target = 45
```

Step	Index	Value	Comparison
1	0	10	10 != 45
2	1	25	25 != 45
3	2	30	30 != 45
4	3	45	FOUND

The search stops at index **3** because continuing would only add unnecessary comparisons.

Linear Search — Complexity

Case	Complexity	When?
Best	$O(1)$	Target is first
Average	$O(n)$	Often scans a portion
Worst	$O(n)$	Last element or absent
Space	$O(1)$	Iterative search

When is Linear Search a good choice?

When the data is unsorted, the collection is small, or you only need a one-off scan. It is also an excellent baseline before optimizing.

INTERVIEW HABIT: Start with the simplest correct solution, then explain what extra structure would enable a faster one.

Binary Search — Introduction

Binary Search repeatedly divides a **sorted / ordered** search space into halves.

IMPORTANT: Classic binary search requires the array to be sorted in the direction your comparisons assume.

```
[2, 4, 6, 8, 10, 12, 14]
```

```
Target = 10
```

```
Compare → Eliminate half → Repeat
```

The key mental model

At every step, the comparison with the middle tells us which half cannot contain the target. That is the source of the speedup.

Why Binary Search Is Faster

Linear Search: potentially checks one element after another.

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Binary Search: repeatedly halves the remaining search space.

8 elements
↓ 4 elements
↓ 2 elements
↓ 1 element

Why $O(\log n)$? Each operation cuts the problem roughly in half. The number of halvings needed to reach one item grows very slowly as n grows.

For large sorted datasets, this difference becomes enormous.

Binary Search — Algorithm

1. Set **left** and **right** boundaries
2. Calculate **mid**
3. Compare `arr[mid]` with target
4. Equal → found
5. Target greater → move left to `mid + 1`
6. Target smaller → move right to `mid - 1`
7. Stop when `left > right`

Search-space view

```
[ L . . M . . R ]  
target > arr[M]  
↓  
discard LEFT half  
  
[ . . . . M . R ]
```

The exact boundary update is crucial: it must discard the middle element after a failed comparison, otherwise an infinite loop can occur.

Binary Search — C++ Implementation

```
int binarySearch(const vector& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1;
}
```

Safe midpoint

```
left + (right - left) / 2
```

This form is preferred over **(left + right) / 2** because it avoids overflow when indices are very large.

Binary Search — Dry Run

```
Array = [2, 4, 6, 8, 10, 12, 14, 16]  
Target = 14
```

Iteration 1

left = 0, right = 7, mid = 3 → arr[mid] = 8. Since 14 > 8, discard the left half.

Iteration 2

left = 4, right = 7, mid = 5 → arr[mid] = 12. Since 14 > 12, move right again.

Iteration 3

left = 6, right = 7, mid = 6 → arr[mid] = 14 → **FOUND at index 6.**

Notice how only three comparisons were needed instead of scanning all eight elements.

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Sorted required	No	Yes / ordered search space
Strategy	Sequential scan	Divide & conquer
Best	$O(1)$	$O(1)$
Average	$O(n)$	$O(\log n)$
Worst	$O(n)$	$O(\log n)$
Space	$O(1)$	$O(1)$ iterative
Difficulty	Very easy	Moderate

Quick Rule: Unsorted → Linear Search. Sorted → Think Binary Search.

Searching with Duplicates

```
[2, 4, 4, 4, 7, 9]  
Target = 4
```

“Find 4” can mean several different requirements:

- Any occurrence
- First occurrence
- Last occurrence
- Lower bound
- Upper bound
- Insertion position

A basic binary search may return **any** matching index. When the problem asks for a boundary, we change what we do after finding a match: record the answer, then continue toward the required side.

This is the foundation of boundary-based binary search.

Binary Search Patterns

Pattern 1 — Search in a Sorted Array

Find whether a target exists and return its position.

Pattern 2 — Search for a Boundary

Find first/last occurrence, lower bound, upper bound, or insertion position.

Pattern 3 — Binary Search on Answer

Search for the smallest/largest feasible answer when a condition is monotonic.

The deeper idea: Binary Search is a problem-solving pattern, not just an array algorithm.

Whenever you can define an ordered search space and a monotonic true/false condition, binary search may be possible.

Interview Problem 1 — Binary Search

Goal: Given a sorted array and a target, return the target's index; otherwise return -1.

```
nums = [-1,0,3,5,9,12]
target = 9
answer = 4
```

Approach

Maintain a closed interval [left, right]. Compare the middle value and eliminate one half.

```
int search(vector& nums, int target) {
    int left = 0, right = nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Complexity: $O(\log n)$ time, $O(1)$ extra space.

Interview Problem 2 — Search Insert Position

```
nums = [1, 3, 5, 6]
```

```
Target = 5 → 2
```

```
Target = 2 → 1
```

Idea

We are looking for the first index where **nums[i] >= target**. That is a lower-bound condition.

```
int searchInsert(vector& nums, int target) {
    int left = 0, right = nums.size();

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid;
    }
    return left;
}
```

Complexity: **O(log n)** time, **O(1)** extra space. Interview insight: boundary problems often use a half-open interval [left, right).

Interview Problem 3 — Search in Rotated Sorted Array

```
[4,5,6,7,0,1,2]
Target = 0
Output = 4
```

Key observation

Even after rotation, at least one side of mid is sorted.

Algorithm

1. Find mid.
2. Identify which half is sorted.
3. Check whether target lies inside that sorted half.
4. Search it if yes; otherwise search the other half.

```
int search(vector& nums, int target) {
    int l = 0, r = nums.size() - 1;

    while (l <= r) {
        int m = l + (r - l) / 2;
        if (nums[m] == target) return m;

        if (nums[l] <= nums[m]) { // left sorted
            if (nums[l] <= target && target < nums[m]) r = m - 1;
            else l = m + 1;
        } else { // right sorted
            if (nums[m] < target && target <= nums[r]) l = m + 1;
            else r = m - 1;
        }
    }
    return -1;
}
```

Complexity: $O(\log n)$ time, $O(1)$ extra space.

Bonus — First and Last Position

```
nums = [5,7,7,8,8,10]
target = 8
output = [3,4]
```

Use two boundary searches:

First occurrence: find the leftmost index with value $\geq$ target.

Last occurrence: find the rightmost index with value $\leq$ target.

A practical implementation uses two helper functions:

```
int firstPos(const vector& a, int x) {
    int l = 0, r = a.size() - 1, ans = -1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] >= x) { if (a[m] == x) ans = m; r = m - 1; }
        else l = m + 1;
    }
    return ans;
}
```

For the last position, mirror the logic by moving left/right in the opposite direction. Complexity: $O(\log n)$.

Interview Tips & Common Mistakes

7 habits to build

1. Always ask: “Is the array sorted?”
2. Clarify duplicate behavior.
3. Define your binary-search invariant.
4. Use the safe midpoint formula.
5. Test empty, one-element, first, last, and absent cases.
6. State complexity before coding.
7. Do not blindly memorize templates—understand the boundary meaning.

Common mistakes

Wrong left/right boundaries • Infinite loops • Incorrect boundary updates • Forgetting empty arrays • Mishandling duplicates • Applying binary search to unsorted data

Best interview move: explain what each pointer represents before writing the loop.

Real-World Example — Product Catalog

Imagine a product catalog whose IDs are stored in sorted order:

```
101 118 135 149 163 177 190 205
```

Search for product ID 177.

Instead of checking every ID, compare against the middle. If the target is larger, everything smaller than the middle can be discarded. Repeat until the target is isolated.

If the catalog were unsorted, those comparisons would not tell us which half can be discarded, so a normal binary search would not be valid.

Real-world lesson: sorted information creates leverage. It turns a scan into a logarithmic search.

Complete Revision Cheat Sheet

Linear Search	Binary Search
$O(n)$	$O(\log n)$
Works on unsorted data	Needs sorted / ordered search space
Sequential scan	Eliminate half
Simple baseline	Powerful interview pattern

Decision tree

```
Is the search space sorted / ordered / monotonic?  
|  
YES  
↓  
Think Binary Search  
|  
NO  
↓  
Consider Linear Search
```

Remember the pattern: Search space → Compare → Eliminate → Repeat.

Day 8 — Takeaway

“Searching is not just about finding an element. It is about reducing unnecessary work.”

Key lessons

- Start with Linear Search.
- Understand when Binary Search applies.
- Learn boundary handling.
- Recognize binary-search patterns.
- Practice on real interview problems.
- Always analyze time and space complexity.

Interview Tip

Before writing Binary Search, clearly define what left, right, and mid represent.

Tomorrow

Day 9 — Introduction to Sorting

From finding elements efficiently to arranging them efficiently — tomorrow we begin Sorting.

DSA From Scratch to Advanced • C++ • Interview Preparation